云电脑容器管理系统 - 完整设计文档

版本信息

- **版本**: v1.4
- **日期**: 2026-02-13
- **状态**: 设计完成
- **更新记录**:
  - v1.4 (2026-02-13): 容器创建自动启动、余额日志完善、客户端复制功能、数值安全处理
  - v1.3 (2026-02-11): 余额变动日志、日志导出、菜单权限调整
  - v1.1 (2026-02-11): 增强管理员管理功能、改进数据隔离、优化日志系统

12. 更新记录

v1.4 (2026-02-13)

新增功能

1. 容器创建自动启动

- 创建云电脑后自动启动实例，无需手动点击启动
- 创建接口返回完整的连接信息（地址、端口、用户名、密码）
- 用户可立即使用远程桌面连接

2. 创建用户时记录余额变动日志

- 管理员创建用户并设置初始余额时，自动记录双方余额变动
- 管理员账户记录 `deduct` 类型日志（扣除余额）
- 用户账户记录 `recharge` 类型日志（初始充值）
- 日志中包含完整信息：变动金额、前后余额、操作人、备注

3. 客户端连接信息一键复制

- 地址、用户名、密码、UHost ID 均支持一键复制到剪贴板
- 连接信息卡片宽度优化（限制最大500px）
- 登录界面标签优化为"手机号-账号"

功能优化

1. 会话统计计算优化

- `calculate_session_stats` 方法增加数据库刷新操作
- 确保获取最新的 `started_at` 时间，避免计算偏差
- 方法签名更新，需要传入 `db` 参数

2. 数值安全处理

- 运行时长、消费金额、余额等数值统一使用 `max(0, value)` 处理
- 防止显示负数，提升用户体验
- 涉及位置：客户端状态显示、停止确认弹窗、账单统计

3. RDP用户名统一

- 所有位置统一使用 "Administrator"（首字母大写）
- 修复 Windows 凭据管理器因大小写问题导致的登录失败
- 涉及文件：config.py、rdp_helper.py、main.py

Bug修复

- 修复定时扣费任务中金额可能为负数的问题
- 修复容器创建后状态统计不准确的问题

涉及文件清单

模块	文件路径	变更类型
后端API	`backend/app/api/admin_user.py`	新增余额日志记录
后端API	`backend/app/api/container.py`	创建后自动启动
后端服务	`backend/app/services/crud_service.py`	刷新数据优化
后端任务	`backend/app/tasks/scheduler.py`	数值安全处理
前端路由	`admin-frontend/src/router/index.js`	菜单顺序调整
客户端配置	`client/config.py`	用户名大小写
客户端主程序	`client/main.py`	复制功能、数值处理
客户端工具	`client/utils/rdp_helper.py`	用户名大小写

v1.3 (2026-02-11)

新增功能

1. 余额变动日志

- 新增余额变动日志标签页，与容器操作日志并列显示
- 支持按账户类型（管理员/用户）、变动类型（充值/扣费/消费/退款）、时间范围筛选
- 显示账户名称、变动金额（红绿色区分）、前后余额、来源、操作人、备注
- 管理员给用户充值时，自动记录双方余额变动日志

2. 日志导出功能

- 容器操作日志支持导出CSV格式
- 余额变化日志支持导出CSV格式
- 导出文件名包含日期，便于管理

3. 菜单权限调整

- 普通管理员菜单：仪表盘、用户管理、日志中心
- 超级管理员菜单：仪表盘、用户管理、管理员管理、日志中心、系统配置

功能优化

1. 仪表盘优化

- "今日收入"卡片改为"剩余可开通用户数"
- 实时显示管理员剩余可开通的用户数量

2. 充值流程优化

- 普通管理员给用户充值时，自动扣除管理员余额
- 充值操作同时记录用户和管理员双方的余额变动日志
- 余额变动日志记录完整信息：变动类型、金额、前后余额、来源、操作人

问题修复

- 移除登录界面默认账号提示信息
- 修复充值接口余额日志未记录的问题
- 修复 `update_balance` 事务提交时机问题
- 完善 `BalanceLog` 数据模型定义

数据库变更

新增表：

```sql
CREATE TABLE balance_log (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    account_type VARCHAR(20) NOT NULL COMMENT '账户类型：admin/user',
    account_id INTEGER NOT NULL COMMENT '账户ID',
    change_type VARCHAR(50) NOT NULL COMMENT '变动类型：recharge/deduct/consume/refund',
    amount FLOAT NOT NULL COMMENT '变动金额',
    balance_before FLOAT NOT NULL COMMENT '变动前余额',
    balance_after FLOAT NOT NULL COMMENT '变动后余额',
    source VARCHAR(50) COMMENT '来源：manual/system/auto',
    related_id INTEGER COMMENT '关联ID（如订单ID、容器ID等）',
    remark TEXT COMMENT '备注',
    operator_id INTEGER COMMENT '操作人ID',
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,

    FOREIGN KEY (operator_id) REFERENCES m_admin(id)
);

CREATE INDEX idx_balance_log_account ON balance_log(account_type, account_id);
CREATE INDEX idx_balance_log_type ON balance_log(change_type);
CREATE INDEX idx_balance_log_created ON balance_log(created_at);
```

API变更

新增接口：

- `GET /api/admin/balance-logs` - 获取余额变动日志列表

筛选参数：

参数	类型	说明
page	int	页码
page_size	int	每页数量
account_type	string	账户类型（admin/user）
change_type	string	变动类型（recharge/deduct/consume/refund）
start_date	string	开始日期（YYYY-MM-DD）
end_date	string	结束日期（YYYY-MM-DD）

响应数据：

```
{
    "code": 200,
    "data": {
        "items": [
            {
                "id": 1,
                "account_type": "user",
                "account_id": 5,
                "account_name": "ABC公司",
                "change_type": "recharge",
                "amount": 100.00,
                "balance_before": 50.00,
                "balance_after": 150.00,
                "source": "manual",
                "operator_name": "代理A",
                "remark": "管理员充值",
                "created_at": "2026-02-11 10:00:00"
            }
        ],
        "total": 100,
        "page": 1,
        "page_size": 20
    }
}
```


前端变更

日志页面（Logs.vue）

- 重构为双标签页结构
- 容器操作日志标签页
- 余额变化日志标签页
- 各标签页独立的筛选条件和分页
- 新增导出CSV功能

菜单配置（Layout.vue）

```
// 超级管理员菜单
const superAdminMenu = [
    { path: '/dashboard', title: '仪表盘', icon: 'Odometer' },
    { path: '/users', title: '用户管理', icon: 'User' },
    { path: '/admins', title: '管理员管理', icon: 'UserFilled' },
    { path: '/logs', title: '日志中心', icon: 'Document' },
    { path: '/config', title: '系统配置', icon: 'Setting' }
]

// 普通管理员菜单
const adminMenu = [
    { path: '/dashboard', title: '仪表盘', icon: 'Odometer' },
    { path: '/users', title: '用户管理', icon: 'User' },
    { path: '/logs', title: '日志中心', icon: 'Document' }
]
```

涉及文件清单

模块	文件路径	变更类型
后端模型	`backend/app/models/models.py`	新增 BalanceLog 模型
后端API	`backend/app/api/admin_balance_log.py`	新增余额日志API
后端API	`backend/app/api/admin_user.py`	修复日志记录
后端API	`backend/app/api/admin_manager.py`	修复日志记录
后端路由	`backend/app/main.py`	注册余额日志路由
数据库脚本	`backend/init_db.py`	BalanceLog表初始化
前端API	`admin-frontend/src/api/index.js`	新增 getBalanceLogs
前端页面	`admin-frontend/src/views/Logs.vue`	重构双标签页
前端菜单	`admin-frontend/src/views/Layout.vue`	菜单权限调整
文档	`CHANGELOG.md`	版本更新日志

v1.1 (2026-02-11)

新增功能

- **管理员增强管理**
  - 管理员支持公司名称、联系人、手机号字段
  - 管理员支持余额账户和用户上限设置
  - 超级管理员可为管理员充值/扣减余额
  - 管理员列表显示用户数和余额信息
- **数据隔离增强**
  - 仪表盘数据按当前管理员隔离，显示"我的用户"、"我的容器"、"我的余额"
  - 用户列表支持按归属管理员筛选（超级管理员）
  - 日志支持按管理员筛选
- **日志系统优化**
  - 容器日志增加归属管理员追踪（admin_name字段）
  - 增强日志详情：启动时间、停止时间、使用时长、费用
  - 支持按操作类型筛选（create/start/stop/delete/auto_stop）
- **错误处理增强**
  - 前端错误详情弹窗显示
  - 支持错误类型分类（validation_error、business_error等）
  - 请求ID追踪便于问题排查

Bug修复

- 解决容器重建时的唯一约束冲突问题

数据库变更

```sql
-- 管理员表新增字段
ALTER TABLE m_admin ADD COLUMN company_name VARCHAR(100);
ALTER TABLE m_admin ADD COLUMN contact_name VARCHAR(50);
ALTER TABLE m_admin ADD COLUMN phone VARCHAR(20);
ALTER TABLE m_admin ADD COLUMN balance DECIMAL(10,2) DEFAULT 0.00;
ALTER TABLE m_admin ADD COLUMN max_users INTEGER DEFAULT 10;

-- 容器日志表新增字段
ALTER TABLE container_log ADD COLUMN admin_id INTEGER;
ALTER TABLE container_log ADD COLUMN admin_name VARCHAR(50);
ALTER TABLE container_log ADD COLUMN started_at DATETIME;
ALTER TABLE container_log ADD COLUMN stopped_at DATETIME;
ALTER TABLE container_log ADD COLUMN duration_minutes INTEGER;
ALTER TABLE container_log ADD COLUMN cost DECIMAL(10,4);
```

API变更

- `GET /api/admin/dashboard` - 返回当前管理员隔离的数据
- `GET /api/admin/admins` - 新增返回balance、user_count、company_name等字段
- `POST /api/admin/admins/{admin_id}/balance` - 新增管理员余额充值接口
- `GET /api/admin/users` - 超级管理员支持admin_id筛选参数
- `GET /api/admin/logs/container` - 新增筛选参数和增强日志字段

目录

1. 项目概述
2. 核心业务逻辑
3. 数据库设计
4. API接口设计
5. 业务流程详解
6. 后台管理功能
7. 客户端设计
8. 定时任务
9. 项目结构
10. 关键业务规则
11. 余额变动日志
12. 更新记录

1. 项目概述

1.1 项目核心特点

特性	说明
容器管理	基于UCloud的GPU云电脑服务
按需计费	按实际运行时长计费，每分钟自动扣费
1:1绑定	一个用户绑定一个云电脑实例
持久化实例	关机后保留实例状态和数据
固定配置	统一的硬件配置（3080Ti, 12核, 32GB, 200GB）
日志记录	完整的操作日志和扣费记录追踪
权限管理	超级管理员 + 管理员 + 用户三级权限

1.2 技术栈

模块	技术
后端（服务端）	Python + FastAPI + SQLAlchemy + SQLite
后台管理前端	Vue 3 + Vite + Element Plus
本地客户端	Python + PySide6
认证	JWT（PyJWT）
云服务商	UCloud SDK

1.3 核心概念

- **用户**: 使用云电脑的企业客户，1:1绑定一个云电脑实例
- **超级管理员**: 系统最高权限，可管理所有管理员、设置价格、查看所有数据
- **管理员**: 可管理用户数据、充值余额、查看日志
- **云电脑实例**: UCloud提供的GPU云主机，按运行时长计费
- **固定配置**: 所有实例统一使用3080Ti GPU, 12核CPU, 32GB内存, 200GB SSD存储
- **每分钟价格**: 由超级管理员设置的系统全局价格（元/分钟）

2. 核心业务逻辑

2.1 绑定关系

```
用户(m_user) ←──1:1──→ 云电脑实例(container_record)
```

- 一个用户只能拥有一个非删除状态的云电脑实例
- 创建新实例前必须先删除旧实例
- 用户通过 `current_container_id` 字段关联当前实例

2.2 实例生命周期

```
┌──────────────────────────────────────────────────────────────────────┐
│                              实例状态流转图                              │
└──────────────────────────────────────────────────────────────────────┘

创建实例
    ↓
已停止(stopped) ←──────┐
    ↓                  │
启动                   │
    ↓                  │
运行中(running) ───────┤
    ↓                  │
停止                   │
    ↓                  │
已停止(stopped) ───────┘
    ↓
删除实例
    ↓
已删除(deleted)
```

状态说明:

- **creating**: 创建中，等待UCloud返回
- **stopped**: 已停止，实例保留，数据不丢失，不计费
- **running**: 运行中，正常计费
- **stopping**: 停止中

- **starting**: 启动中
- **deleting**: 删除中
- **deleted**: 已删除，数据丢失
- **error**: 异常状态

2.3 计费模式

项目	说明
计费粒度	按分钟计费
扣费频率	每分钟自动扣费一次（后台定时任务）
价格设置	超级管理员在系统配置中设置全局统一价
启动门槛	余额必须 >= 5分钟费用（防止频繁启停）
余额不足	余额<=0时立即自动停止实例
价格快照	创建实例时记录当时的每分钟价格，防止中途调价

3. 数据库设计

3.1 用户表 (m_user)

```sql
CREATE TABLE m_user (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    company_name VARCHAR(100) NOT NULL COMMENT '公司名称',
    contact_name VARCHAR(50) NOT NULL COMMENT '联系人姓名',
    phone VARCHAR(20) NOT NULL UNIQUE COMMENT '联系电话（登录账号）',
    password_hash VARCHAR(255) NOT NULL COMMENT '密码哈希（bcrypt）',
    balance DECIMAL(10,2) DEFAULT 0.00 COMMENT '账号余额（元）',
    current_container_id INTEGER DEFAULT NULL COMMENT '当前绑定的容器ID',
    status INTEGER DEFAULT 1 COMMENT '状态: 0-禁用, 1-正常',
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    last_login_at DATETIME DEFAULT NULL,

    FOREIGN KEY (current_container_id) REFERENCES container_record(id)
);

CREATE INDEX idx_user_phone ON m_user(phone);
CREATE INDEX idx_user_status ON m_user(status);
```

3.2 管理员表 (m_admin)

```sql
CREATE TABLE m_admin (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    username VARCHAR(50) NOT NULL UNIQUE COMMENT '管理员账号',
    password_hash VARCHAR(255) NOT NULL COMMENT '密码哈希',
    role VARCHAR(20) NOT NULL DEFAULT 'admin' COMMENT '角色: super_admin/admin',
    status INTEGER DEFAULT 1 COMMENT '状态: 0-禁用, 1-正常',
    company_name VARCHAR(100) DEFAULT NULL COMMENT '公司名称',
    contact_name VARCHAR(50) DEFAULT NULL COMMENT '联系人姓名',
    phone VARCHAR(20) DEFAULT NULL COMMENT '联系电话',
    balance DECIMAL(10,2) DEFAULT 0.00 COMMENT '账号余额（元）',
    max_users INTEGER DEFAULT 10 COMMENT '最大用户数上限',
    created_by INTEGER DEFAULT NULL COMMENT '创建者ID（超级管理员）',
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    last_login_at DATETIME DEFAULT NULL,
    last_login_ip VARCHAR(50) DEFAULT NULL,

    FOREIGN KEY (created_by) REFERENCES m_admin(id)
);

CREATE INDEX idx_admin_username ON m_admin(username);
CREATE INDEX idx_admin_role ON m_admin(role);
```

3.3 容器记录表 (container_record)

```sql
CREATE TABLE container_record (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER NOT NULL UNIQUE COMMENT '所属用户ID（1:1绑定）',
    ucloud_instance_id VARCHAR(100) NOT NULL COMMENT 'UCloud实例ID',
    instance_name VARCHAR(100) NOT NULL COMMENT '实例名称',
    status VARCHAR(20) NOT NULL DEFAULT 'creating' COMMENT '状态',
    gpu_type VARCHAR(50) NOT NULL DEFAULT '3080Ti' COMMENT 'GPU类型（固定）',
    cpu_cores INTEGER NOT NULL DEFAULT 12 COMMENT 'CPU核数（固定）',
    memory_gb INTEGER NOT NULL DEFAULT 32 COMMENT '内存大小GB（固定）',
    storage_gb INTEGER NOT NULL DEFAULT 200 COMMENT '存储大小GB（固定）',
    price_per_minute DECIMAL(10,4) NOT NULL COMMENT '创建时的每分钟价格',
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    started_at DATETIME DEFAULT NULL COMMENT '本次启动时间（用于计费）',
    stopped_at DATETIME DEFAULT NULL COMMENT '本次停止时间',
    deleted_at DATETIME DEFAULT NULL,
    total_running_minutes INTEGER DEFAULT 0 COMMENT '累计运行分钟数',
    total_cost DECIMAL(10,2) DEFAULT 0.00 COMMENT '累计消费金额',

    FOREIGN KEY (user_id) REFERENCES m_user(id),
    UNIQUE(user_id)
);

CREATE INDEX idx_container_user ON container_record(user_id);
CREATE INDEX idx_container_status ON container_record(status);
CREATE INDEX idx_container_ucloud_id ON container_record(ucloud_instance_id);
```

硬件配置说明（固定）：

- Zone: cn-wlcb-01
- MachineType: G
- CompShareImageId: compshareImage-1mnqn08rd1xz
- GPU: 1 x 3080Ti
- CPU: 12核
- 内存: 32GB
- 存储: 200GB SSD

3.4 系统配置表 (system_config)

```sql
CREATE TABLE system_config (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    config_key VARCHAR(50) NOT NULL UNIQUE COMMENT '配置项键名',
    config_value VARCHAR(255) NOT NULL COMMENT '配置项值',
    description TEXT COMMENT '配置描述',
    updated_by INTEGER DEFAULT NULL COMMENT '更新者ID',
    updated_at DATETIME DEFAULT CURRENT_TIMESTAMP,

    FOREIGN KEY (updated_by) REFERENCES m_admin(id)
);

-- 初始化数据
INSERT INTO system_config (config_key, config_value, description) VALUES
('price_per_minute', '0.5000', '云电脑每分钟价格（元）'),
('min_balance_to_start', '2.5000', '启动云电脑所需最低余额（5分钟费用）'),
('auto_stop_threshold', '0.0000', '自动停止余额阈值');
```

3.5 实时扣费记录表 (billing_charge_record)

```sql
CREATE TABLE billing_charge_record (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER NOT NULL COMMENT '用户ID',
    container_id INTEGER NOT NULL COMMENT '容器ID',
    charge_minute DATETIME NOT NULL COMMENT '扣费的这一分钟（精确到分钟）',
    price_per_minute DECIMAL(10,4) NOT NULL COMMENT '当时每分钟价格',
    amount DECIMAL(10,4) NOT NULL COMMENT '扣费金额',
    balance_before DECIMAL(10,2) NOT NULL COMMENT '扣费前余额',
    balance_after DECIMAL(10,2) NOT NULL COMMENT '扣费后余额',
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,

    FOREIGN KEY (user_id) REFERENCES m_user(id),
    FOREIGN KEY (container_id) REFERENCES container_record(id)
);

CREATE INDEX idx_charge_user ON billing_charge_record(user_id);
CREATE INDEX idx_charge_container ON billing_charge_record(container_id);
CREATE INDEX idx_charge_minute ON billing_charge_record(charge_minute);
```

3.6 余额变动记录表 (balance_log)

```sql
CREATE TABLE balance_log (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    account_type VARCHAR(20) NOT NULL COMMENT '账户类型：admin/user',
    account_id INTEGER NOT NULL COMMENT '账户ID',
    change_type VARCHAR(50) NOT NULL COMMENT '变动类型：recharge/deduct/consume/refund',
    amount FLOAT NOT NULL COMMENT '变动金额',
    balance_before FLOAT NOT NULL COMMENT '变动前余额',
    balance_after FLOAT NOT NULL COMMENT '变动后余额',
    source VARCHAR(50) COMMENT '来源：manual/system/auto',
    related_id INTEGER COMMENT '关联ID（如容器ID）',
    remark TEXT COMMENT '备注',
    operator_id INTEGER COMMENT '操作人ID',
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,

    FOREIGN KEY (operator_id) REFERENCES m_admin(id)
);

CREATE INDEX idx_balance_log_account ON balance_log(account_type, account_id);
CREATE INDEX idx_balance_log_type ON balance_log(change_type);
CREATE INDEX idx_balance_log_created ON balance_log(created_at);
```

说明：此表记录所有账户（管理员和用户）的余额变动，采用 account_type + account_id 联合标识账户。

3.7 容器操作日志表 (container_log)

```sql
CREATE TABLE container_log (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER NOT NULL COMMENT '操作用户ID',
    container_id INTEGER NOT NULL COMMENT '容器ID',
    admin_id INTEGER DEFAULT NULL COMMENT '操作管理员ID（管理员操作时记录）',
    admin_name VARCHAR(50) DEFAULT NULL COMMENT '管理员名称',
    action VARCHAR(50) NOT NULL COMMENT '操作：create/start/stop/delete/connect/auto_stop',
    action_status VARCHAR(20) NOT NULL COMMENT '状态：success/failed/pending',
    request_data TEXT COMMENT '请求参数（JSON）',
    response_data TEXT COMMENT '响应数据（JSON）',
    error_message TEXT COMMENT '错误信息',
    ip_address VARCHAR(50) COMMENT '操作IP',
    user_agent TEXT COMMENT '客户端信息',
    started_at DATETIME DEFAULT NULL COMMENT '容器启动时间',
    stopped_at DATETIME DEFAULT NULL COMMENT '容器停止时间',
    duration_minutes INTEGER DEFAULT NULL COMMENT '使用时长（分钟）',
    cost DECIMAL(10,4) DEFAULT NULL COMMENT '本次操作产生的费用',
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,

    FOREIGN KEY (user_id) REFERENCES m_user(id),
    FOREIGN KEY (container_id) REFERENCES container_record(id),
    FOREIGN KEY (admin_id) REFERENCES m_admin(id)
);

CREATE INDEX idx_container_log_user ON container_log(user_id);
CREATE INDEX idx_container_log_container ON container_log(container_id);
CREATE INDEX idx_container_log_admin ON container_log(admin_id);
```

3.8 管理员操作日志表 (admin_operation_log)

```sql
CREATE TABLE admin_operation_log (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    admin_id INTEGER NOT NULL COMMENT '操作管理员ID',
    action VARCHAR(50) NOT NULL COMMENT '操作类型',
    target_type VARCHAR(50) NOT NULL COMMENT '操作对象类型',
    target_id INTEGER COMMENT '操作对象ID',
    old_value TEXT COMMENT '修改前数据（JSON）',
    new_value TEXT COMMENT '修改后数据（JSON）',
    description TEXT COMMENT '操作描述',
    ip_address VARCHAR(50),
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,

    FOREIGN KEY (admin_id) REFERENCES m_admin(id)
);

CREATE INDEX idx_admin_log_admin ON admin_operation_log(admin_id);
```

4. API接口设计

4.1 认证相关

用户登录

```http
POST /api/auth/login
Content-Type: application/json

{
    "phone": "13800138000",
    "password": "user_password"
}
```

管理员登录

```
POST /api/auth/admin/login
Content-Type: application/json

{
    "username": "admin",
    "password": "admin_password"
}
```

4.2 用户相关（客户端）

获取当前用户信息

```
GET /api/user/me
Authorization: Bearer {token}
```

获取余额和消费统计

```
GET /api/user/statistics
Authorization: Bearer {token}
```

4.3 容器相关（客户端）

获取我的云电脑

```
GET /api/container/my
Authorization: Bearer {token}
```

获取实时状态

```
GET /api/container/my/status
Authorization: Bearer {token}
```

创建云电脑

```
POST /api/container
Authorization: Bearer {token}
Content-Type: application/json

{
    "instance_name": "云电脑-ABC公司"
}

响应:
{
    "code": 200,
    "message": "云电脑创建中",
    "data": {
        "container_id": 1,
        "status": "creating",
        "estimated_time": 120
    }
}
```

说明: 云电脑使用固定配置创建，无需选择硬件参数：

- GPU: NVIDIA 3080Ti x 1
- CPU: 12核
- 内存: 32GB
- 存储: 200GB SSD

错误响应:

```
{
    "code": 400,
    "message": "您已有一个云电脑实例，请先删除后再创建",
    "error_code": "CONTAINER_ALREADY_EXISTS"
}
```

启动云电脑

```
POST /api/container/start
Authorization: Bearer {token}
```

停止云电脑

```
POST /api/container/stop
Authorization: Bearer {token}
```

删除云电脑

```
DELETE /api/container
Authorization: Bearer {token}

请求体:
{
    "confirm": true
}
```

获取连接信息

```
GET /api/container/connection
Authorization: Bearer {token}
```

4.4 账单相关（客户端）

获取扣费记录

```
GET /api/billing/charges?page=1&page_size=20
Authorization: Bearer {token}
```

获取余额变动记录

```
GET /api/billing/balance-logs?page=1&page_size=20
Authorization: Bearer {token}
```

4.5 系统配置（超级管理员）

获取系统配置

```
GET /api/admin/config
Authorization: Bearer {admin_token}
```

更新每分钟价格

```
PUT /api/admin/config/price
Authorization: Bearer {admin_token}
Content-Type: application/json

{
    "price_per_minute": 0.60
}
```

4.6 用户管理（管理员）

获取用户列表

```
GET /api/admin/users?page=1&page_size=20&keyword=&status=&admin_id=
Authorization: Bearer {admin_token}
```

说明:

- 超级管理员可使用 `admin_id` 参数筛选指定管理员的用户
- 普通管理员只能看到自己创建的用户

创建用户

```
POST /api/admin/users
Authorization: Bearer {admin_token}
Content-Type: application/json

{
    "company_name": "XYZ公司",
    "contact_name": "李四",
    "phone": "13900139000",
    "password": "initial_password",
    "initial_balance": 0.00
}
```

充值余额

```
POST /api/admin/users/{user_id}/balance
Authorization: Bearer {admin_token}
Content-Type: application/json

{
    "type": "recharge",
    "amount": 100.00,
    "description": "春节活动赠送"
}
```

删除用户

```
DELETE /api/admin/users/{user_id}
Authorization: Bearer {super_admin_token}
```

说明:

- 仅超级管理员可执行删除操作
- 如果用户存在云电脑实例，则不允许删除（需先删除云电脑）

错误响应:

```
{
    "code": 400,
    "message": "该用户存在云电脑，请先删除云电脑后再删除用户"
}
```

4.7 管理员管理（超级管理员）

获取管理员列表

```
GET /api/admin/admins?page=1&page_size=20
Authorization: Bearer {super_admin_token}
```

响应数据:

```
{
    "code": 200,
    "data": {
        "items": [
            {
                "id": 1,
                "username": "代理A",
                "role": "admin",
                "status": 1,
                "balance": 500.00,
                "max_users": 10,
                "user_count": 5,
                "company_name": "A科技公司",
                "contact_name": "张经理",
                "phone": "13800138001",
                "created_at": "2026-02-10 10:00",
                "last_login_at": "2026-02-11 09:00",
                "created_by": "admin"
            }
        ],
        "total": 1,
        "page": 1,
        "page_size": 20
    }
}
```

创建管理员

```
POST /api/admin/admins
Authorization: Bearer {super_admin_token}
Content-Type: application/json

{
    "username": "代理A",
    "password": "password123",
    "role": "admin",
    "company_name": "A科技公司",
    "contact_name": "张经理",
    "phone": "13800138001",
    "initial_balance": 0.00,
    "max_users": 10
}
```

管理员余额充值

```
POST /api/admin/admins/{admin_id}/balance
Authorization: Bearer {super_admin_token}
Content-Type: application/json

{
    "type": "recharge",
    "amount": 1000.00,
    "description": "代理商充值"
}
```

响应数据:

```
{
    "code": 200,
    "message": "操作成功",
    "data": {
        "admin_id": 2,
        "balance_before": 500.00,
        "balance_after": 1500.00,
        "change_amount": 1000.00
    }
}
```

4.8 日志管理（管理员）

获取容器操作日志

```
GET /api/admin/logs/container?page=1&page_size=20&action=&user_id=&admin_id=
Authorization: Bearer {admin_token}
```

响应数据:


```
{
    "code": 200,
    "data": {
        "items": [
            {
                "id": 1,
                "user_name": "ABC公司",
                "admin_name": "代理A",
                "action": "create",
                "action_status": "success",
                "started_at": "2026-02-11 10:00:00",
                "stopped_at": null,
                "duration_minutes": null,
                "cost": null,
                "ip_address": "192.168.1.100",
                "created_at": "2026-02-11 10:00:00"
            }
        ],
        "total": 100,
        "page": 1,
        "page_size": 20
    }
}
```


筛选参数:

- `action`：操作类型（create/start/stop/delete/auto_stop）
- `user_id`：指定用户ID
- `admin_id`：归属管理员ID（仅超级管理员可用）

4.9 仪表盘统计（管理员）

获取系统概览

```
GET /api/admin/dashboard
Authorization: Bearer {admin_token}
```

说明: 仪表盘数据按当前管理员隔离，显示该管理员的数据

响应数据:

```
{
    "code": 200,
    "data": {
        "overview": {
            "total_users": 5,
            "total_containers": 5,
            "running_containers": 3,
            "stopped_containers": 2,
            "total_balance": 500.00,
            "remaining_users": 5
        },
        "statistics": {
            "today_new_users": 1,
            "today_new_containers": 1,
            "today_running_minutes": 120,
            "today_income": 50.00,
            "this_month_new_users": 5,
            "this_month_income": 500.00
        },
        "charts": {
            "income_trend": [
                {"date": "2026-02-05", "income": 100.00},
                {"date": "2026-02-06", "income": 150.00}
            ],
            "container_status": {
                "running": 3,
                "stopped": 2
            }
        }
    }
}
```

4.10 错误处理增强

错误响应格式

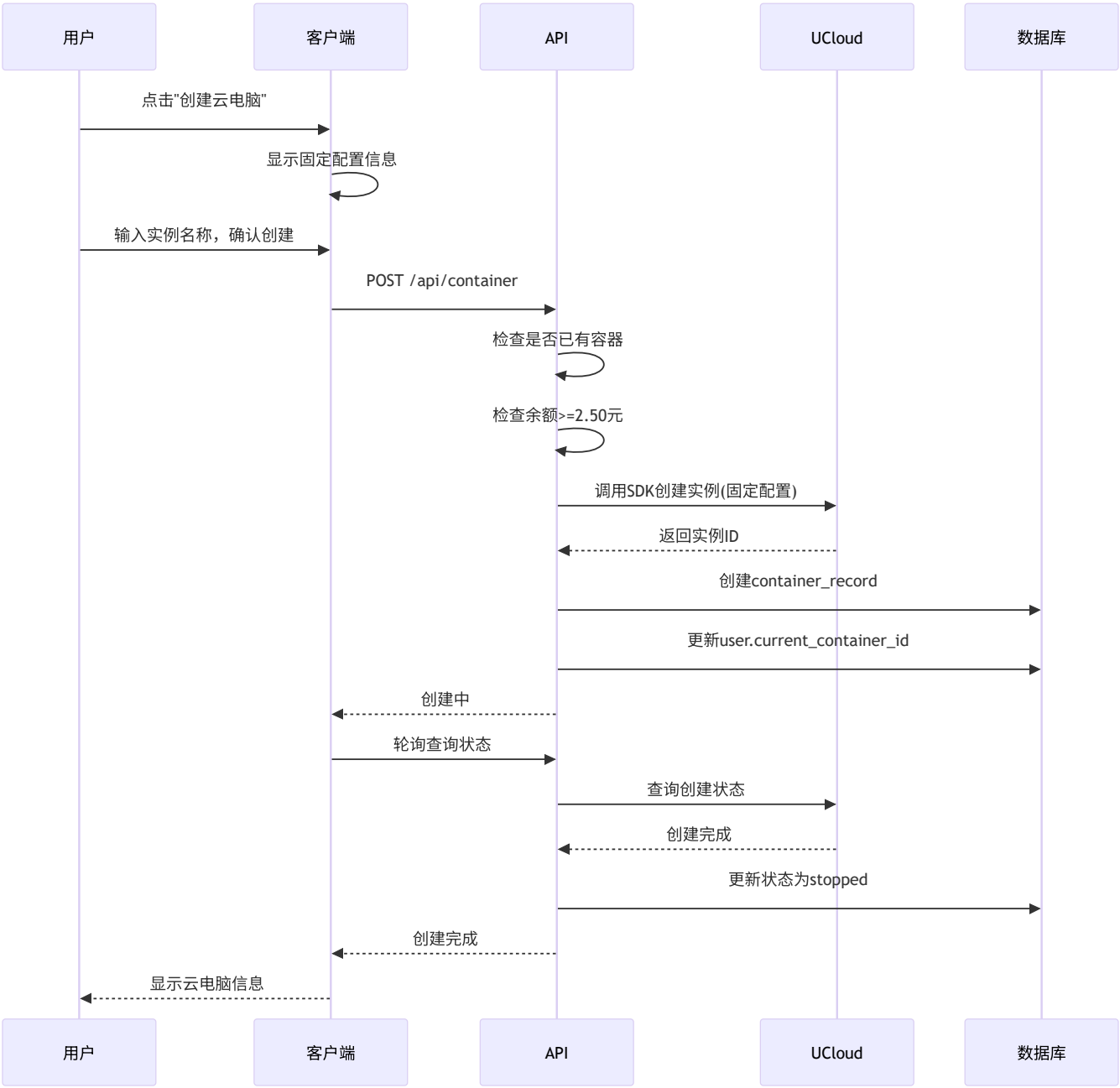
```
{
    "code": 400,
    "message": "请求参数错误",
    "request_id": "req_abc123",
    "error_type": "validation_error",
    "detail": {
        "error_class": "ValueError",
        "error_message": "金额必须大于0",
        "traceback": [
            "File ... in handle_recharge",
            "ValueError: 金额必须大于0"
        ]
    }
}
```

错误类型说明:

error_type	说明
validation_error	请求参数验证错误
business_error	业务逻辑错误
authentication_error	认证错误
authorization_error	权限错误
network_error	网络连接错误
timeout	请求超时
ucloud_error	UCloud API错误

5. 业务流程详解

5.1 创建云电脑流程

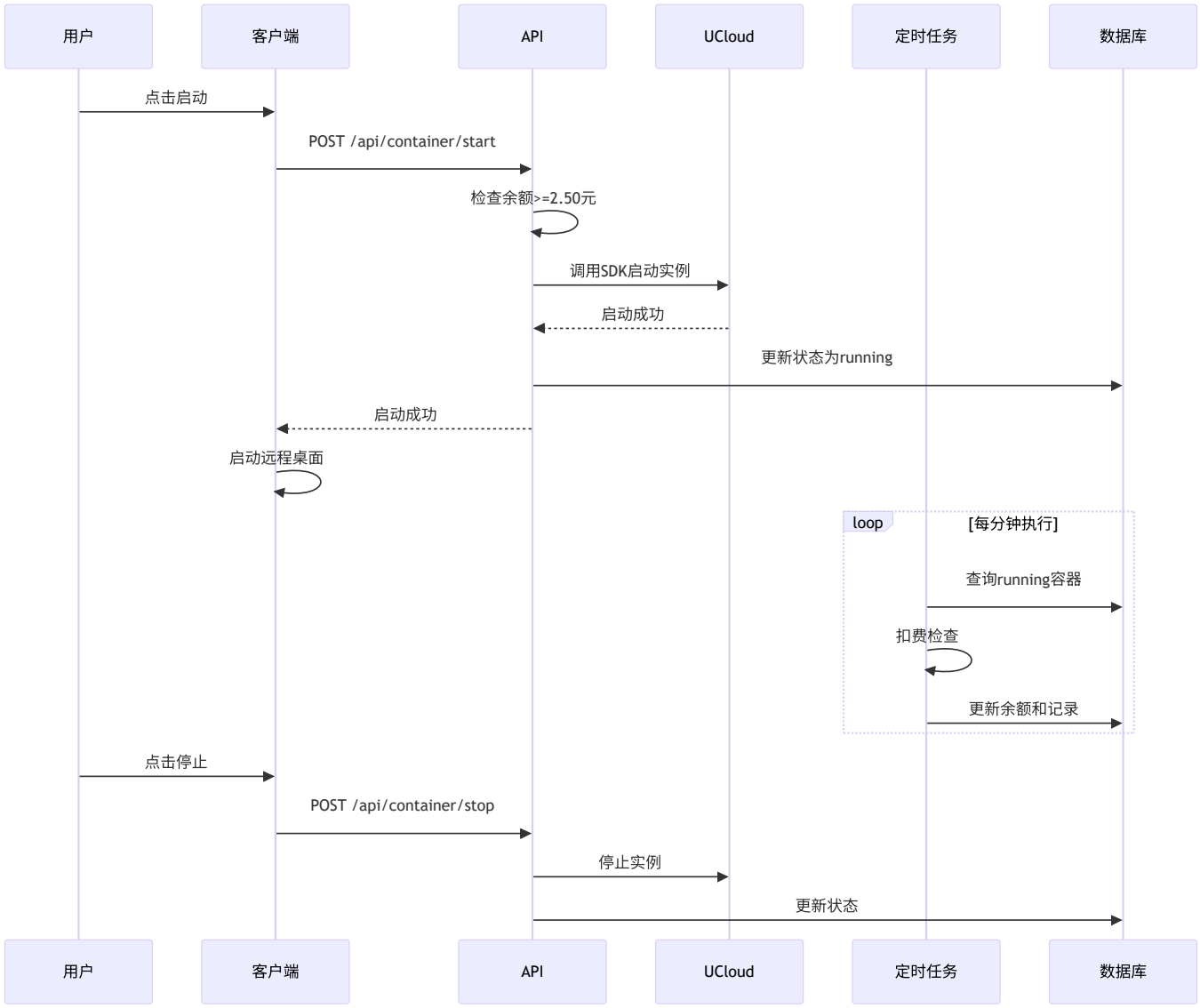


固定配置参数:

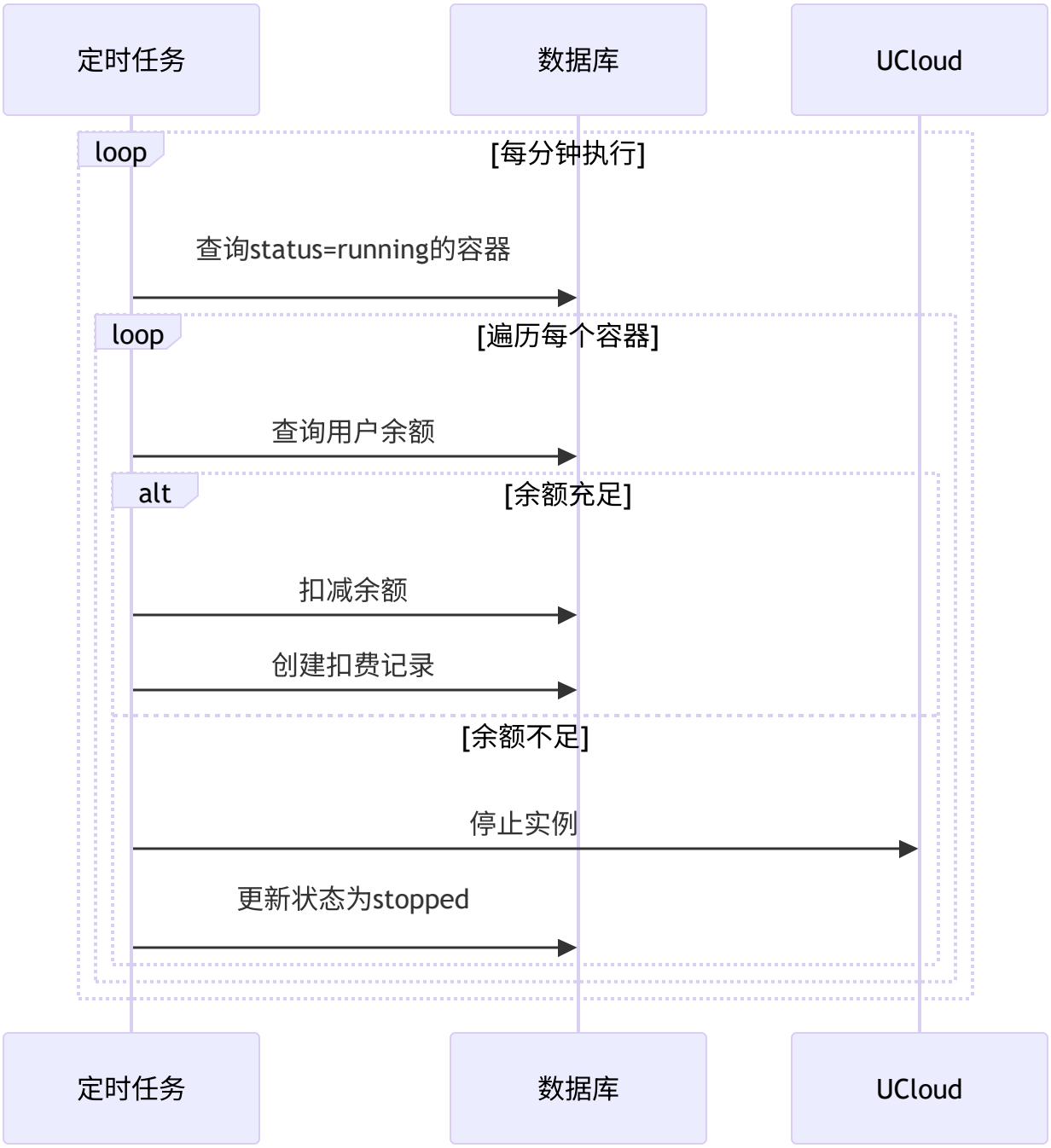
- Zone: cn-wlcb-01
- MachineType: G
- CompShareImageId: compshareImage-1mnqn08rd1xz
- GPU: 1 x 3080Ti
- CPU: 12核
- Memory: 32768MB (32GB)

- Disk: 200GB SSD

5.2 日常使用流程



5.3 每分钟扣费流程



6. 后台管理功能

6.1 仪表盘

- 用户总数、容器总数、运行中容器数
- 我的余额、剩余可开通用户数

- 近7天收入趋势图
- 容器状态分布

6.2 用户管理

- 用户列表（搜索、筛选、分页）
- 创建/编辑用户
- 余额充值/扣减
- 重置用户密码
- 删除用户（仅超级管理员，需先删除云电脑）
- 查看用户详情和消费统计
- 强制停止运行中的容器

6.3 管理员管理（仅超级管理员）

- 管理员列表（显示公司名、联系人、手机号、余额、用户数）
- 创建/编辑管理员（支持设置公司信息、初始余额、用户上限）
- 管理员余额充值/扣减
- 重置密码
- 禁用/启用账号
- 查看管理员创建的用户数量

6.4 系统配置（仅超级管理员）

- 每分钟价格设置
- 启动最低余额设置
- 配置修改历史

6.5 日志中心

- **容器操作日志**
  - 支持按用户、管理员、操作类型筛选
  - 日志显示：启动时间、停止时间、使用时长、费用
  - 归属管理员追踪
  - 支持导出CSV格式
- **余额变动日志**
  - 支持按账户类型（管理员/用户）、变动类型筛选
  - 支持按时间范围筛选
  - 显示：账户名称、变动金额（红绿色区分）、前后余额、来源、操作人、备注

    - 管理员给用户充值时，自动记录双方余额变动
    - 支持导出CSV格式
- 管理员操作日志
- 扣费记录查询

7. 客户端设计

7.1 架构设计

```
client/
├── main.py              # 应用入口
├── api/
│   └── client.py       # HTTP客户端
└── ui/                 # 界面组件
```

7.2 实时状态更新

- 每60秒轮询一次状态
- 实时显示余额和剩余时间
- 余额不足预警（<5分钟时提示）

8. 定时任务

8.1 每分钟扣费任务

```python
async def minute_billing_task():
    """每分钟扣费任务"""
    async with get_db() as db:
        running_containers = await get_running_containers(db)

        for container in running_containers:
            user = await get_user(db, container.user_id)
            price = await get_current_price(db)

            if user.balance < price:
                # 余额不足，停止实例
                await stop_instance(container)
            else:
                # 执行扣费
                await charge_fee(user, container, price)
```

8.2 其他定时任务

- 清理过期日志（每天凌晨3点）
- 生成日报（每天凌晨1点）

9. 项目结构

```
cloud-pc-management/
├── backend/                    # FastAPI后端
│   ├── app/
│   │   ├── api/              # API路由
│   │   ├── models/           # SQLAlchemy模型
│   │   ├── services/         # 业务逻辑
│   │   ├── tasks/            # 定时任务
│   │   └── core/             # 核心配置
│   ├── requirements.txt
│   └── run.py
│
├── admin-frontend/             # Vue3后台管理
│   ├── src/
│   │   ├── api/             # API接口
│   │   ├── views/           # 页面组件
│   │   └── stores/          # Pinia状态
│   └── package.json
│
├── client/                     # PySide6客户端
│   ├── api/
│   ├── ui/
│   └── main.py
│
├── test_api.py               # API测试
├── test_container_api.py     # 容器API测试
├── test_delete_user.py       # 删除用户API测试
├── test_client.py            # 客户端测试
└── AGENTS.md                 # 开发规范
```

10. 关键业务规则

10.1 计费规则

规则项	说明
计费粒度	按分钟计费

规则项	说明
扣费频率	每分钟自动扣费
价格快照	创建时记录价格，不受后续调价影响
启动门槛	余额 >= 2.50元（5分钟费用）
自动停止	余额不足时自动停止

10.2 容器配置规则

配置项	固定值	说明
Zone	cn-wlcb-01	乌兰察布一区
MachineType	G	GPU机型
Image	compshareImage-1mnqn08rd1xz	共享镜像
GPU	3080Ti x 1	NVIDIA显卡
CPU	12核	固定12核
内存	32GB	固定32GB
存储	200GB SSD	系统盘

10.3 权限规则

角色	权限
用户	管理自己的云电脑、查看账单
管理员	管理用户、充值、查看日志（日志中心）、重置用户密码
超级管理员	所有权限 + 管理管理员 + 系统配置 + 删除用户

10.4 状态流转规则

- stopped → running: 启动（检查余额）
- running → stopped: 停止或余额不足
- stopped → deleted: 删除

11. 余额变动日志

11.1 功能概述

余额变动日志用于记录所有账户（管理员、用户）的余额变化情况，支持完整的资金流动追踪。

11.2 变动类型

类型	说明	金额正负
recharge	充值	正数
deduct	扣减/扣费	负数
consume	消费	负数
refund	退款	正数

11.3 余额变动场景

11.3.1 超级管理员给管理员充值

```
超级管理员 ──充值──→ 管理员
记录：
- 管理员账户：recharge（+金额）
```

11.3.2 普通管理员给用户充值

```
管理员 ──扣除──→ 管理员账户
管理员 ──充值──→ 用户账户
记录：
- 管理员账户：deduct（-金额）
- 用户账户：recharge（+金额）
```

11.3.3 容器自动扣费

```
用户账户 ──扣费──→ 系统
记录：
- 用户账户：consume（-金额）
```

11.4 日志字段说明

字段	类型	说明
id	int	日志ID
account_type	string	账户类型（admin/user）
account_id	int	账户ID
account_name	string	账户名称（查询时关联）
change_type	string	变动类型
amount	float	变动金额
balance_before	float	变动前余额
balance_after	float	变动后余额
source	string	来源（manual/system/auto）
related_id	int	关联ID（如容器ID）
remark	string	备注
operator_id	int	操作人ID
operator_name	string	操作人名称（查询时关联）
created_at	datetime	创建时间

11.5 API接口

获取余额变动日志列表

```
GET /api/admin/balance-logs?page=1&page_size=20&account_type=&change_type=&start_date=&end_date=
Authorization: Bearer {admin_token}
```

响应数据


```
{
    "code": 200,
    "data": {
        "items": [
            {
                "id": 1,
                "account_type": "user",
                "account_id": 5,
                "account_name": "ABC公司",
                "change_type": "recharge",
                "amount": 100.00,
                "balance_before": 50.00,
                "balance_after": 150.00,
                "source": "manual",
                "related_id": null,
                "operator_name": "代理A",
                "remark": "管理员充值",
                "created_at": "2026-02-11T10:00:00"
            },
            {
                "id": 2,
                "account_type": "admin",
                "account_id": 2,
                "account_name": "代理A",
                "change_type": "deduct",
                "amount": -100.00,
                "balance_before": 500.00,
                "balance_after": 400.00,
                "source": "manual",
                "related_id": 5,
                "operator_name": "代理A",
                "remark": "给用户 ABC公司 充值",
                "created_at": "2026-02-11T10:00:00"
            }
        ],
        "total": 100,
        "page": 1,
        "page_size": 20
    }
}
```

11.6 前端展示

筛选条件

- 账户类型：管理员/用户
- 变动类型：充值/扣费/消费/退款
- 时间范围：开始日期 ~ 结束日期

表格列

- ID
- 账户名称
- 账户类型（Tag标签）
- 变动类型（带颜色标签）
- 变动金额（正数绿色，负数红色）
- 变动前余额
- 变动后余额
- 来源（Tag标签）
- 操作人
- 备注
- 时间

导出功能

- 支持导出当前筛选条件下的所有数据
- 导出格式：CSV
- 文件名： `余额变动日志_YYYY-MM-DD.csv`

附录

A. 状态码定义

状态码	说明
200	成功
400	请求参数错误
401	未认证

状态码	说明
403	无权限
404	资源不存在
402	余额不足
500	服务器错误

B. 错误码定义

错误码	说明
CONTAINER_ALREADY_EXISTS	已存在容器实例
INSUFFICIENT_BALANCE	余额不足
CONTAINER_NOT_FOUND	容器不存在
INVALID_STATUS	状态不允许此操作
UCLOUD_API_ERROR	UCloud API调用失败

C. 默认账号

- **超级管理员**: admin / Admin123@
- **管理员**: 由超级管理员创建
- **用户**: 由管理员创建